

```
// BRAIN LAYER. Holds the
conversation and talks to whichever
provider it is given.
// Later versions will add tool
routing and memory inside respond().

import { JarvisError } from
"./errors.js";

export const DEFAULT_SYSTEM_PROMPT =
  "You are JARVIS, a friendly voice
  assistant. Your answers are read
  aloud, so reply in plain " +
  "spoken language, usually one to
  three short sentences. No markdown,
  no lists, no emojis. " +
  "If the user just greets you, greet
  them back briefly and ask how you can
  help.";

export class Brain {
  constructor(provider, {
    systemPrompt = DEFAULT_SYSTEM_PROMPT,
    maxTurns = 10 } = {}) {
    this.provider = provider;
    this.systemPrompt = systemPrompt;
```

```
        this.maxTurns = maxTurns;
        this.history = []; // short-term
memory: recent user/assistant
messages
    }

    reset() {
        this.history = [];
    }

    async respond(userText, { signal }
= {}) {
        const text = (userText ||
        "").trim();
        if (!text) throw new
JarvisError("no_speech");
        const pending = [...this.history,
{ role: "user", content: text }];
        const messages = [{ role:
"system", content: this.systemPrompt
}, ...pending];
        const reply = await
this.provider.chat(messages, { signal
});
        if (signal && signal.aborted)
throw new JarvisError("aborted");
        // Only remember the exchange if
it fully succeeded.
        this.history = [...pending, {
```

```
role: "assistant", content: reply  
}].slice(-this.maxTurns * 2);  
    return reply;  
}  
}
```